

Cross-Platform Libc-Free C I/O Library

<https://gemini.google.com/app/ff2f0f42d97a5101>

User prompt: Is it possible writing a pure C library (that can be compiled to dll/so/dylib that contains some functions that do not use libc? The function should be using only OS native code to and support OSX/windows/linux. The function needer are: - getchar (without waiting for the user to press enter): to get a character from the keyboard - printchar: given a file ID it prints that character

Response: **Yes, it is fully possible** to write a pure C library (.dll, .so, .dylib) without linking against libc. Doing so requires bypassing standard runtime initializers and issuing direct kernel interfaces (system calls) or OS platform APIs.

Key Technical Requirements

- **Compiler Flags:** Compiling without libc requires telling the toolchain to omit standard libraries and entry points (e.g., `-nostdlib -ffreestanding -shared` for GCC/Clang or `/NODEFAULTLIB` for MSVC).
- **OS Interoperability Strategy:**
 - **Windows:** Windows kernel system call numbers change between OS updates. Bypassing libc on Windows means importing `kernel32.dll` via `GetStdHandle`, `ReadFile`, and `WriteFile` rather than making raw `syscall` instructions.
 - **Linux:** Issues direct `syscall` assembly instructions (`sys_read` for 0, `sys_write` for 1).
 - **macOS:** Bypasses `libSystem` using macOS x86_64 / ARM64 `syscall` conventions or linking `libSystem.dylib` directly without standard libc runtime wrappers.

Cross-Platform Implementation

```
/* platform_io.h */
#ifndef PLATFORM_IO_H
#define PLATFORM_IO_H

#if defined(_WIN32)
#define EXPORT __declspec(dllexport)
#else
#define EXPORT __attribute__((visibility("default")))
#endif

EXPORT char get_char_no_wait(void);
EXPORT void print_char(int fd, char c);

#endif

/* platform_io.c */
#include "platform_io.h"

// =====
// WINDOWS IMPLEMENTATION (Using kernel32.dll without C Runtime / libc)
// =====
#if defined(_WIN32)

typedef void* HANDLE;
typedef unsigned long DWORD;
typedef int BOOL;

#define STD_INPUT_HANDLE ((DWORD)-10)
#define STD_OUTPUT_HANDLE ((DWORD)-11)
#define STD_ERROR_HANDLE ((DWORD)-12)

#define ENABLE_LINE_INPUT 0x0002
#define ENABLE_ECHO_INPUT 0x0004

// Import Windows kernel primitives directly without CRT linking
__declspec(dllimport) HANDLE __stdcall GetStdHandle(DWORD nStdHandle);
__declspec(dllimport) BOOL __stdcall GetConsoleMode(HANDLE hConsoleHandle, DWORD* lpMode);
__declspec(dllimport) BOOL __stdcall SetConsoleMode(HANDLE hConsoleHandle, DWORD dwMode);
__declspec(dllimport) BOOL __stdcall ReadFile(HANDLE hFile, void* lpBuffer, DWORD nNumberOfBytesToRead,
__declspec(dllimport) BOOL __stdcall WriteFile(HANDLE hFile, const void* lpBuffer, DWORD nNumberOfBytesT

EXPORT char get_char_no_wait(void) {
    HANDLE hStdin = GetStdHandle(STD_INPUT_HANDLE);
    DWORD mode = 0, bytesRead = 0;
    char c = 0;

    // Temporarily disable canonical line buffering and character echoing
    GetConsoleMode(hStdin, &mode);
    SetConsoleMode(hStdin, mode & ~(ENABLE_LINE_INPUT | ENABLE_ECHO_INPUT));

    ReadFile(hStdin, &c, 1, &bytesRead, 0);

    // Restore original console mode
    SetConsoleMode(hStdin, mode);
    return c;
}
```

```

EXPORT void print_char(int fd, char c) {
    DWORD handle_id;
    if (fd == 1)        handle_id = STD_OUTPUT_HANDLE;
    else if (fd == 2)    handle_id = STD_ERROR_HANDLE;
    else                 handle_id = (DWORD)fd;

    HANDLE hFile = GetStdHandle(handle_id);
    DWORD bytesWritten = 0;
    WriteFile(hFile, &c, 1, &bytesWritten, 0);
}

// =====
// LINUX IMPLEMENTATION (Pure Direct Kernel Syscalls)
// =====
#elif defined(__linux__)

#define SYS_READ  0
#define SYS_WRITE 1
#define SYS_IOCTL 16

#define TCGETS 0x5401
#define TCSETS 0x5402
#define ICANON 0000002
#define ECHO 0000010

struct termios {
    unsigned int c_iflag;
    unsigned int c_oflag;
    unsigned int c_cflag;
    unsigned int c_lflag;
    unsigned char c_line;
    unsigned char c_cc[32];
    unsigned int c_ispeed;
    unsigned int c_ospeed;
};

static long syscall3(long num, long a1, long a2, long a3) {
    long ret;
    __asm__ __volatile__(
        "syscall"
        : "=a"(ret)
        : "a"(num), "D"(a1), "S"(a2), "d"(a3)
        : "rcx", "r11", "memory"
    );
    return ret;
}

EXPORT char get_char_no_wait(void) {
    struct termios oldt, newt;
    char c = 0;

    // Disable canonical mode & echo via raw ioctl system call
    syscall3(SYS_IOCTL, 0, TCGETS, (long)&oldt);
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    syscall3(SYS_IOCTL, 0, TCSETS, (long)&newt);

    // sys_read(fd=0, buf=&c, count=1)
    syscall3(SYS_READ, 0, (long)&c, 1);

    // Restore terminal flags
    syscall3(SYS_IOCTL, 0, TCSETS, (long)&oldt);
    return c;
}

EXPORT void print_char(int fd, char c) {
    // sys_write(fd, buf=&c, count=1)
    syscall3(SYS_WRITE, fd, (long)&c, 1);
}

// =====
// MACOS IMPLEMENTATION (x86_64 / ARM64 Direct Syscalls)
// =====
#elif defined(__APPLE__)

#define SYSCALL_CLASS_SHIFT 24
#define SYSCALL_CLASS_UNIX 2
#define SYSCALL_CONSTRUCT(class, code) (((class) << SYSCALL_CLASS_SHIFT) | (code))

#define SYS_READ  SYSCALL_CONSTRUCT(SYSCALL_CLASS_UNIX, 3)
#define SYS_WRITE SYSCALL_CONSTRUCT(SYSCALL_CLASS_UNIX, 4)
#define SYS_IOCTL SYSCALL_CONSTRUCT(SYSCALL_CLASS_UNIX, 54)

```

```

#define TIOCGETA 0x40487413
#define TIOCSETA 0x80487414
#define ICANON   0x00000100
#define ECHO     0x00000008

struct termios {
    unsigned long c_iflag;
    unsigned long c_oflag;
    unsigned long c_cflag;
    unsigned long c_lflag;
    unsigned char c_cc[20];
    unsigned long c_ispeed;
    unsigned long c_ospeed;
};

static long syscall3(long num, long a1, long a2, long a3) {
    long ret;
#ifdef __x86_64__
    __asm__ __volatile__ (
        "syscall"
        : "=a"(ret)
        : "a"(num), "D"(a1), "S"(a2), "d"(a3)
        : "rcx", "r11", "memory"
    );
#elif defined(__aarch64__)
    register long x16 __asm__ ("x16") = num;
    register long x0  __asm__ ("x0")  = a1;
    register long x1  __asm__ ("x1")  = a2;
    register long x2  __asm__ ("x2")  = a3;
    __asm__ __volatile__ (
        "svc #0x80"
        : "=r"(x0)
        : "r"(x16), "r"(x0), "r"(x1), "r"(x2)
        : "memory"
    );
    ret = x0;
#endif
    return ret;
}

EXPORT char get_char_no_wait(void) {
    struct termios oldt, newt;
    char c = 0;

    syscall3(SYS_IOCTL, 0, TIOCGETA, (long)&oldt);
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    syscall3(SYS_IOCTL, 0, TIOCSETA, (long)&newt);

    syscall3(SYS_READ, 0, (long)&c, 1);

    syscall3(SYS_IOCTL, 0, TIOCSETA, (long)&oldt);
    return c;
}

EXPORT void print_char(int fd, char c) {
    syscall3(SYS_WRITE, fd, (long)&c, 1);
}

#endif

```

Compilation Commands (No `libc`)

Linux (`libplatform_io.so`)

```
gcc -O2 -shared -fPIC -nostdlib -ffreestanding platform_io.c -o libplatform_io.so
```

macOS (`libplatform_io.dylib`)

```
clang -O2 -dynamiclib -nostdlib -ffreestanding platform_io.c -o libplatform_io.dylib
```

Windows (`platform_io.dll`)

```
cl /O2 /LD /NODEFAULTLIB platform_io.c kernel32.lib /link /NOENTRY /DLL /OUT:platform_io.dll
```

User prompt: Can the same code be written in Ada?

Response: **Yes, absolutely.** You can implement the exact same low-level, zero-`libc` OS interface library in Ada.

Ada is exceptionally well-suited for standard-library-free systems programming. By using **Zero Footprint (ZFP)** compilation mode (or the `-nolibc` / `-nostdlib` flags), suppressing the standard Ada runtime (RTS), and utilizing `Pragma Import / System.Machine_Code`, Ada can

interact directly with host system calls or platform libraries.

Key Ada Capabilities for Zero-libc OS Interoperability

- **No-RTS / ZFP Mode:** Using `pragma No_Run_Time`; tells the GNAT compiler not to link any standard runtime components or C standard library dependencies.
- **Direct C / Win32 Import:** Ada's `pragma Import (Stdcall, ...)` or `pragma Import (C, ...)` lets you import functions directly from OS system libraries (like Windows `kernel32.dll`) without relying on standard C `stdio.h` or standard headers.
- **Inline Assembly (`System.Machine_Code`):** For POSIX targets (Linux/macOS), Ada provides `System.Machine_Code.Asm` to issue raw `syscall` or `svc` assembly instructions directly to the kernel.

Implementation in Ada

1. Specification (`platform_io.ads`)

```
pragma No_Run_Time;

package Platform_IO is

  -- Exported symbols for dynamic/shared library compilation (.dll / .so / .dylib)
  function Get_Char_No_Wait return Character;
  pragma Export (C, Get_Char_No_Wait, "get_char_no_wait");

  procedure Print_Char (Fd : Integer; C : Character);
  pragma Export (C, Print_Char, "print_char");

end Platform_IO;
```

2. Body (`platform_io.adb`)

```
pragma No_Run_Time;

with System;
with System.Machine_Code; use System.Machine_Code;

package body Platform_IO is

  -- =====
  -- WINDOWS IMPLEMENTATION (Direct imports from kernel32.dll)
  -- =====
  #if Win32 then

    subtype HANDLE is System.Address;
    subtype DWORD is Interfaces.C.unsigned_long;
    subtype BOOL   is Interfaces.C.int;

    STD_INPUT_HANDLE  : constant DWORD := 4294967286; -- (DWORD)-10
    STD_OUTPUT_HANDLE : constant DWORD := 4294967285; -- (DWORD)-11
    STD_ERROR_HANDLE  : constant DWORD := 4294967284; -- (DWORD)-12

    ENABLE_LINE_INPUT : constant DWORD := 16#0002#;
    ENABLE_ECHO_INPUT : constant DWORD := 16#0004#;

    function GetStdHandle (nStdHandle : DWORD) return HANDLE;
    pragma Import (Stdcall, GetStdHandle, "GetStdHandle");

    function GetConsoleMode (hConsoleHandle : HANDLE; lpMode : access DWORD) return BOOL;
    pragma Import (Stdcall, GetConsoleMode, "GetConsoleMode");

    function SetConsoleMode (hConsoleHandle : HANDLE; dwMode : DWORD) return BOOL;
    pragma Import (Stdcall, SetConsoleMode, "SetConsoleMode");

    function ReadFile
      (hFile      : HANDLE;
       lpBuffer   : access Character;
       nNumberOfBytesToRead : DWORD;
       lpNumberOfBytesRead  : access DWORD;
       lpOverlapped : System.Address) return BOOL;
    pragma Import (Stdcall, ReadFile, "ReadFile");

    function WriteFile
      (hFile      : HANDLE;
       lpBuffer   : access Character;
       nNumberOfBytesToWrite : DWORD;
       lpNumberOfBytesWritten : access DWORD;
       lpOverlapped : System.Address) return BOOL;
    pragma Import (Stdcall, WriteFile, "WriteFile");

    function Get_Char_No_Wait return Character is
      HStdin : constant HANDLE := GetStdHandle (STD_INPUT_HANDLE);
```

```

Mode      : aliased DWORD := 0;
Bytes_Read : aliased DWORD := 0;
C         : aliased Character := ASCII.NUL;
Success    : BOOL;
begin
    Success := GetConsoleMode (HStdin, Mode'Access);
    Success := SetConsoleMode (HStdin, Mode and not (ENABLE_LINE_INPUT or ENABLE_ECHO_INPUT));

    Success := ReadFile (HStdin, C'Access, 1, Bytes_Read'Access, System.Null_Address);

    Success := SetConsoleMode (HStdin, Mode);
    return C;
end Get_Char_No_Wait;

procedure Print_Char (Fd : Integer; C : Character) is
    Handle_Id : DWORD;
    HFile     : HANDLE;
    Bytes_Written : aliased DWORD := 0;
    C_Var      : aliased Character := C;
    Success    : BOOL;
begin
    if Fd = 1 then
        Handle_Id := STD_OUTPUT_HANDLE;
    elsif Fd = 2 then
        Handle_Id := STD_ERROR_HANDLE;
    else
        Handle_Id := DWORD (Fd);
    end if;

    HFile := GetStdHandle (Handle_Id);
    Success := WriteFile (HFile, C_Var'Access, 1, Bytes_Written'Access, System.Null_Address);
end Print_Char;

-- =====
-- LINUX IMPLEMENTATION (Pure Direct Kernel Syscalls)
-- =====
#elsif Linux then

    type Termios is record
        C_Iflag : Interfaces.C.unsigned;
        C_Oflag : Interfaces.C.unsigned;
        C_Cflag : Interfaces.C.unsigned;
        C_Lflag : Interfaces.C.unsigned;
        C_Line  : Interfaces.C.unsigned_char;
        C_Cc    : String (1 .. 32);
        C_Ispeed : Interfaces.C.unsigned;
        C_Ospeed : Interfaces.C.unsigned;
    end record;
    pragma Convention (C, Termios);

    SYS_READ  : constant := 0;
    SYS_WRITE : constant := 1;
    SYS_IOCTL : constant := 16;

    TCGETS : constant := 16#5401#;
    TCSETS : constant := 16#5402#;
    ICANON  : constant := 8#0000002#;
    ECHO    : constant := 8#0000010#;

    function Syscall3 (Num, A1, A2, A3 : System.Address) return System.Address is
        Result : System.Address;
    begin
        Asm ("syscall",
            Outputs => System.Address'Output ("a", Result),
            Inputs  => [System.Address'Input ("a", Num),
                       System.Address'Input ("D", A1),
                       System.Address'Input ("S", A2),
                       System.Address'Input ("d", A3)],
            Clobbers => "rcx, r11, memory",
            Volatile => True);
        return Result;
    end Syscall3;

    function Get_Char_No_Wait return Character is
        Oldt, Newt : aliased Termios;
        C          : aliased Character := ASCII.NUL;
        Dummy      : System.Address;
    begin
        Dummy := Syscall3 (System.Storage_Elements.To_Address (SYS_IOCTL),
                           System.Storage_Elements.To_Address (0),
                           System.Storage_Elements.To_Address (TCGETS),
                           Oldt'Address);

        Newt := Oldt;

```

```

Newt.C_Lflag := Newt.C_Lflag and not (ICANON or ECHO);

Dummy := Syscall3 (System.Storage_Elements.To_Address (SYS_IOCTL),
                  System.Storage_Elements.To_Address (0),
                  System.Storage_Elements.To_Address (TCSETS),
                  Newt'Address);

Dummy := Syscall3 (System.Storage_Elements.To_Address (SYS_READ),
                  System.Storage_Elements.To_Address (0),
                  C'Address,
                  System.Storage_Elements.To_Address (1));

Dummy := Syscall3 (System.Storage_Elements.To_Address (SYS_IOCTL),
                  System.Storage_Elements.To_Address (0),
                  System.Storage_Elements.To_Address (TCSETS),
                  Oldt'Address);

return C;
end Get_Char_No_Wait;

procedure Print_Char (Fd : Integer; C : Character) is
  C_Var : aliased Character := C;
  Dummy : System.Address;
begin
  Dummy := Syscall3 (System.Storage_Elements.To_Address (SYS_WRITE),
                    System.Storage_Elements.To_Address (Fd),
                    C_Var'Address,
                    System.Storage_Elements.To_Address (1));

end Print_Char;

#end if;

end Platform_IO;

```

Building Without libc Linkage in GNAT

To build a dynamic shared library (.so, .dylib, or .dll) directly using GNAT without linking standard runtime packages:

```

# Building shared dynamic library on Linux without libc / standard RTS
gnatmake -P platform_io.gpr -XBUILD=Release --RTS=zfp -largs -nostdlib -shared

```

Alternatively, via a **GNAT Project File (.gpr)**:

```

project Platform_IO is
  for Library_Name use "platform_io";
  for Library_Kind use "dynamic";
  for Source_Dirs use ("src");
  for Object_Dir use "obj";
  for Library_Dir use "lib";

  package Compiler is
    for Default_Switches ("Ada") use ("-O2", "-gnatg", "-ffreestanding");
  end Compiler;

  package Linker is
    for Default_Switches ("Ada") use ("-nostdlib");
  end Linker;
end Platform_IO;

```
